Fundamentals of DevOps and Software Delivery
Yevgeniy Brikman
Published by O'Reilly Media, Inc.
59% complete Approx. 6 hours left
Collapse
Contents Highlights
Example: Deploy the Sample App Locally
Deploying an App on a Server
Comparing Deployment Options
The Evolution of DevOps
Adopting DevOps Practices
Conclusion
2. How To Manage Your Infrastructure As Code
100% complete
3. How To Manage Your Apps By Using Orchestration Tools
100% complete
4. How To Version, Build, And Test Your Code
100% complete

Chapter 1. How to Deploy Your App

In the Preface, you read that DevOps consists of dozens of concepts. But it almost always starts with just one question: "I wrote an app. Now what?"

You and your team have spent months putting together an app. You picked a programming language, implemented the backend, designed and built a user interface (UI), and finally, it's time to expose the app to real users. How, exactly, do you do that?

There are so many questions to figure out here. Should you use AWS or Azure? (And what about Heroku or Vercel?) Do you need one server or multiple servers? (Or serverless?) Do you need to use Docker? (Or Kubernetes?) Do you need a VPC? (Or a VPN?) How do you get a domain name? (And what about a TLS certificate?) What's the right way to set up your database? (And how do you back it up?) Why did that app crash? Why does nothing seem to be working? Why is this so hard?

OK, easy now. Take a deep breath. If you're new to software delivery—you've worked as an app developer your whole career, or you're just starting out in operations—it can be overwhelming, and you can get stuck in analysis paralysis. This book is here to help. I will walk you through each of these questions—and many others you didn't think to ask—and help you figure out the answers, step-by-step.

The first step will be to deploy the app on a server and get it running in the most basic way you can. In this chapter, you'll work through examples to deploy the same app on your own computer, on Render (a platform as a service), and on AWS (an infrastructure as a service). After that, you'll see how to evolve your basic deployment as your company grows.

Without further ado, let's jump right in and start deploying some apps!

Example: Deploy the Sample App Locally

The first place you should be able to deploy any app is locally, on your own computer. This is typically how you'd build the app in the first place, writing and running your code locally until it's working. The way you deploy an app locally depends on the technology. Throughout this book, you're going to be using a simple Node.js sample app.

EXAMPLE CODE

As a reminder, you can find all the code examples in the book's repo in GitHub.

Create a new folder on your computer, perhaps called something like *fundamentals-of-devops*, which you can use to store code for the examples you'll be running throughout the book. You can run the following commands in a terminal to create the folder and go into it:

```
$ mkdir fundamentals-of-devops
$ cd fundamentals-of-devops
```

In that folder, create new subfolders for this chapter and the sample app:

```
$ mkdir -p ch1/sample-app
$ cd ch1/sample-app
```

The sample app you'll be using is a minimal "Hello, World" Node.js app, written in JavaScript. You don't need to understand much JavaScript to make sense of the app. One of the nice things about getting started with Node.js is that all the code for a simple web app fits in a single file that's about 10 lines long. Within the *sample-app* folder, create a file called *app.js*, with the contents shown in Example 1-1.

Example 1-1. A Node.js "Hello, World" sample app (*ch1/sample-app/app.js*)

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, World!\n');              ❶
});

const port = process.env.PORT || 8080;  ❷
server.listen(port,() => {
  console.log(`Listening on port ${port}`);
});
```

This "Hello, World" app does the following:

❶ Respond to all requests with a 200 status code and the text `Hello, World!`

❷ Listen for requests on the port number specified via the `PORT` environment variable, or if `PORT` is not set, default to port 8080.

To run the app, you must first install Node.js (minimum version 21). You can then start the app with `node app.js`:

```
$ node app.js
Listening on port 8080
```

Next, open *http://localhost:8080* in your browser, and you should see this:

```
Hello, World!
```

Congrats, you're running the app locally! That's a great start, but if you want to expose your app to users, you'll need to run it on a server, as discussed next.

Deploying an App on a Server

When you run an app on your computer, it is available only on *localhost*, a hostname configured to point to the *loopback network interface*, which means it bypasses any real network interface, and can be accessed *only* from your own computer and not from the outside world. This is by design, and for the most part, a good thing, as the way you run apps on a personal computer for development and testing is *not* the way you should run them when you want to expose them to outsiders.

KEY TAKEAWAY 1

You should never expose apps running on a personal computer to the outside world.

Instead, if you're going to expose your app to the outside world, you should run it on a server. A *server* is a computer specifically designed for running apps and exposing those apps to the outside world. A server and a personal computer differ in many ways, including the following:

Security

Most servers run a stripped-down OS and are *hardened* against attacks (e.g., firewall, intrusion-prevention tools, file-integrity monitoring). Your personal computer has all sorts of extra software (any of which could have a vulnerability) and is not hardened.

Availability

Most servers are designed to be on all the time and have redundant power. Your personal computer may shut off at any time.

Performance

Most servers run just the apps on them. You use your personal computer for other tasks (e.g., coding, browsing) that may impact your app's performance.

Collaboration

Most apps are worked on by teams, and whereas other developers don't (and shouldn't) have access to your personal computer, servers are usually designed for team access.

For these reasons, always use a server to run your production apps. Broadly speaking, you have two ways to get access to servers (two options for *hosting* your apps):

- You can buy and set up your own servers (on premises).
- You can rent servers from others (the cloud).

We'll discuss each of these next.

On-Prem and Cloud Hosting

The traditional way to run software is to buy servers and set them up *on premises* (*on prem* for short), in a physical location you own. When you are just starting out, the location could be as simple as a closet in your office, but as a company grows, so do the computing demands, and you eventually need a *data center*, with all the requisite equipment (e.g., racks, servers, hard drives, cooling) and staff (e.g., electricians, network administrators, security). So for decades, if you wanted to build a software company, you also had to invest quite a bit into hardware.

This started to change in 2006 with the launch of Amazon Web Services (AWS), the first *cloud computing platform* (*cloud* for short), which allowed you to rent servers using a software interface, either via a few clicks (which you'll do in this chapter) or via a few lines of code (which you'll do in Chapter 2). This profound shift let you get up and running in minutes instead of months, at the cost of a few cents (or even free) instead of thousands of dollars.

There are two main cloud offerings: *infrastructure as a service* (IaaS), which gives you access to low-level computing resources (servers, hard drives, networks), and leaves it up to you to put them together into a software delivery process, and *platform as a service* (PaaS), which gives you access to higher-level primitives, including an opinionated software delivery process. To get a feel for the difference, you'll use a PaaS in the next section and an IaaS in the section after that.

Example: Deploy an App via PaaS (Render)

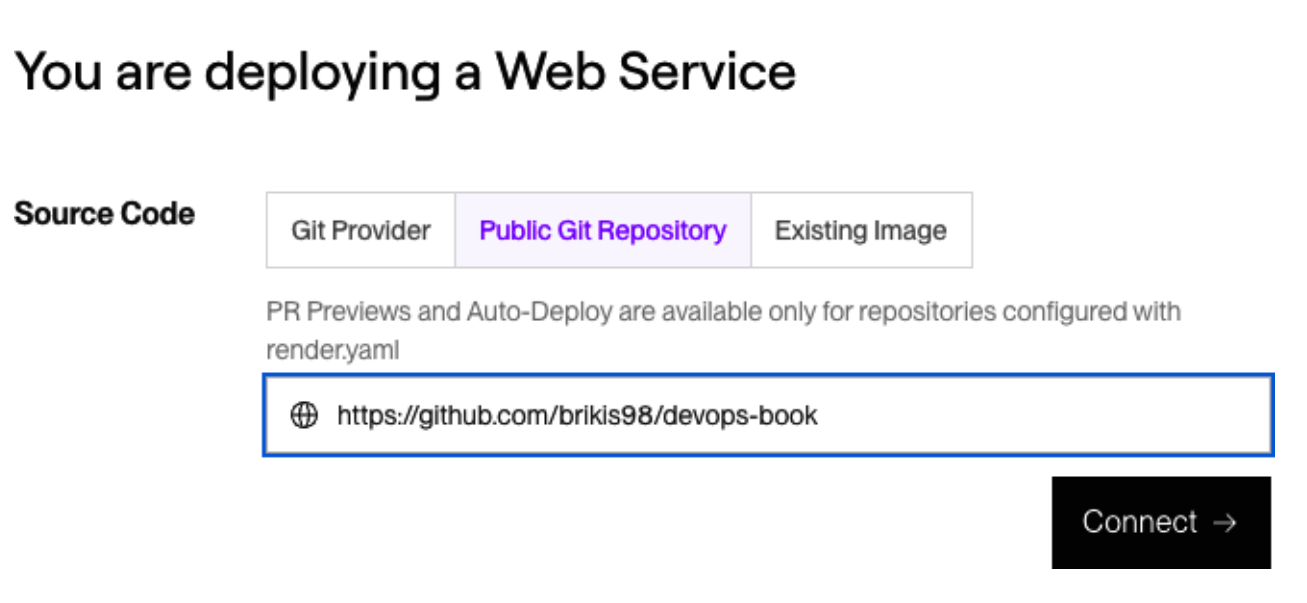
Popular PaaS providers include Heroku, Render, Fly.io, and Vercel (the book's website provides a full list). Heroku was one of the first PaaS providers, and it used to be my go-to choice, but it discontinued its free tier in 2022. Therefore, for the examples in this book, you'll be using Render, which offers a free Hobby tier and supports running apps in many languages and frameworks (including Node.js) without having to set up a build system or framework (topics you'll learn about later in the book). Render also has a good reputation in the community and is often described as the spiritual successor to Heroku. To deploy the sample app by using Render, go through the following steps:

Step 1: Sign up for a Render account

Create a new account on *render.com*.

Step 2: Deploy a new web service

Head to the Render Dashboard and click the Deploy a Web Service button. On the next page, select the Public Git Repository tab and enter the URL of this book's repo, *https://github.com/brikis98/devops-book*, as shown in Figure 1-1. This repo contains the Node.js code from Example 1-1 in the *ch1/sample-app* folder, so this lets you deploy the app without creating your own GitHub repo.



You are deploying a Web Service

PR Previews and Auto-Deploy are available only for repositories configured with render.yaml

https://github.com/brikis98/devops-book

Figure 1-1. Use this book's sample code repo to create a web service in Render

Click the Connect button, and then configure your web service as shown in Table 1-1.

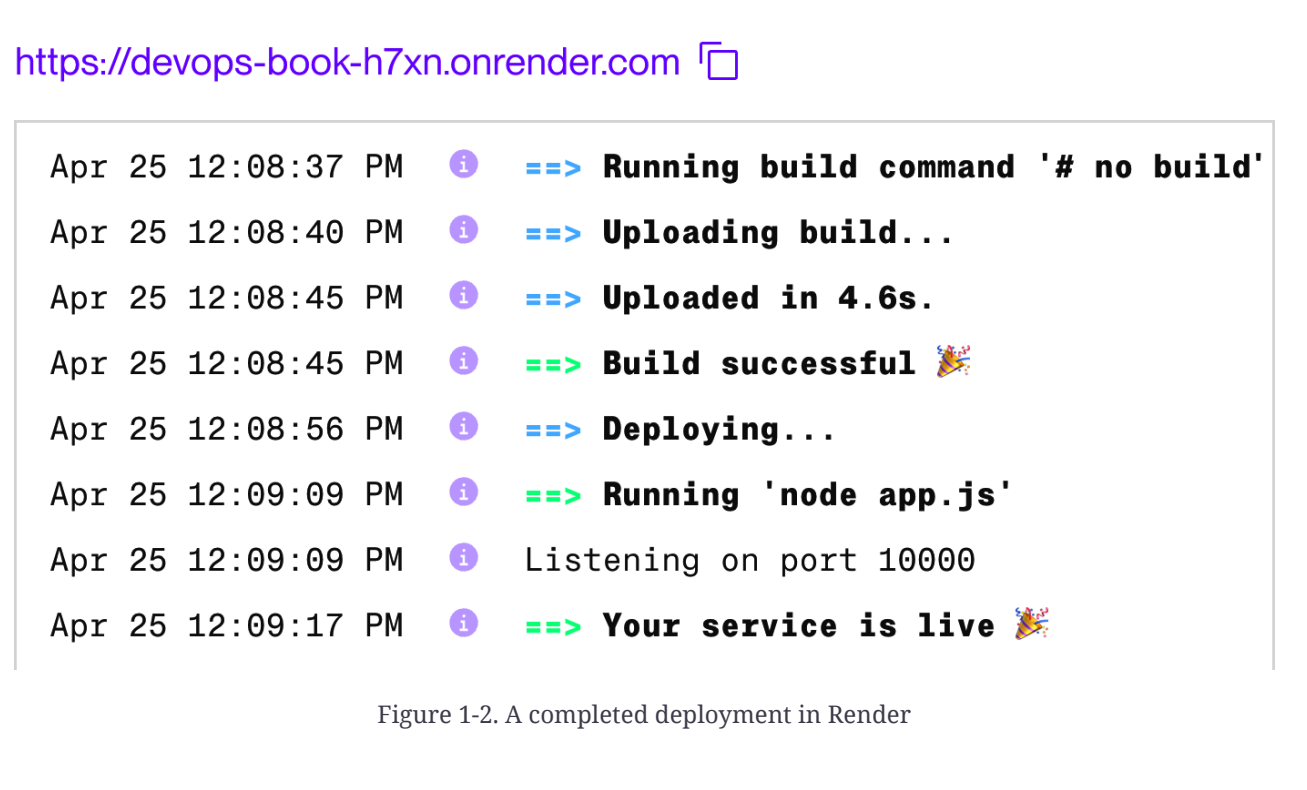
Table 1-1. The values to configure for your web service in Render

Configuration	Value
Name	sample-app
Language	Node
Root Directory	*ch1/sample-app*
Build Command	`# (no build command)`
Start Command	`node app.js`
Instance Type	Free

Leave the other settings (e.g., Project, Branch, Region) at their default values and click the Deploy Web Service button at the bottom of the page to start the deployment.

Step 3: Test your app

As shown Figure 1-2, after a couple of minutes the deployment log should say something like `Your service is live`.



brikis98 / devops-book main

https://devops-book-h7xn.onrender.com

```
Apr 25 12:08:37 PM  ==> Running build command '# no build'
Apr 25 12:08:40 PM  ==> Uploading build...
Apr 25 12:08:45 PM  ==> Uploaded in 4.6s.
Apr 25 12:08:45 PM  ==> Build successful 🎉
Apr 25 12:08:56 PM  ==> Deploying...
Apr 25 12:09:09 PM  ==> Running 'node app.js'
Apr 25 12:09:09 PM  Listening on port 10000
Apr 25 12:09:17 PM  ==> Your service is live 🎉
```

Figure 1-2. A completed deployment in Render

At the top left of the page, you should see a randomly generated URL for your app of the form `https://<NAME>.onrender.com`. Open this URL, and you should see this:

```
Hello, World!
```

Congrats, in just a few steps, you now have an app running on a server!

GET YOUR HANDS DIRTY

Here are a few exercises you can try at home to go deeper:

- Click the Scale tab to change how many servers run your app.
- Click the Logs tab to see the logs for your app.
- Click the Metrics tab to see metrics for your app.

When you're done experimenting with Render, undeploy your app by clicking the Settings tab, scrolling to the bottom, and clicking the Delete Web Service button.

Using a PaaS typically means you get not just a server, but a lot of powerful functionality out of the box: scaling to multiple servers, domain names (*<NAME>.onrender.com*), encryption (HTTPS URLs), monitoring (logs and metrics), and more. This is the power of PaaS: in a matter of minutes, a good PaaS can take care of so many software delivery concerns for you. It's like magic. And that's the greatest strength of PaaS: it just works.

Except when it doesn't. When that happens, this same magic becomes the greatest weakness of PaaS. By design, with a PaaS, just about everything is happening behind the scenes, so if something doesn't work, debugging or fixing it can be hard. Moreover, to make the magic possible, most PaaS offerings have limitations on what you can deploy, what types of apps you can run, what sort of access you can have to the underlying hardware, what sort of hardware is available, and so on. If the PaaS doesn't support it—if the command-line interface (CLI) or UI the PaaS provides doesn't expose the ability to do something you need—you typically can't do it at all.

As a result, while many projects start on PaaS, if they grow big enough and require more control, they end up migrating to IaaS.

Example: Deploy an App via IaaS (AWS)

Broadly speaking, the IaaS space falls into three buckets:

Virtual private server

Some companies primarily focus on giving you access to a *virtual private server* (VPS) for as cheap as possible. These companies might offer a few other features (e.g., networking, storage) as well, but the main reason you'd go with one of these providers is that you just want a replacement for having to rack your own servers, and you prefer to have someone else do it for you and give you access. Some of the big players in this space include Hetzner, DigitalOcean, Vultr, and Akamai Connected Cloud (the book's website provides a full list).

Content delivery networks

Other companies primarily focus on *content delivery networks* (CDNs), which are servers that are distributed all over the world, typically for serving and caching content. Again, these companies might offer a few other features (e.g., protection against attacks), but the main reason you'd go with one of these providers is that your user base is geographically distributed, and you need a fast and reliable way to serve them content with low latency. You'll learn all about CDNs in Chapter 9.

Cloud providers

Finally, a handful of large companies are trying to provide general-purpose cloud solutions that offer everything: VPS, CDN, containers, serverless, data storage, file storage, machine learning, natural language processing, edge computing, and more. The big players in this space include AWS, Google Cloud, and Microsoft Azure (full list).

In general, the VPS and CDN providers are specialists in their respective areas, so in those areas, they will typically beat a general-purpose cloud provider in terms of features, pricing, and user experience. For example, a VPS from Hetzner is usually faster, cheaper, and in some ways easier to use than one from AWS. So if you need *only* those specific items, you're better off going with a specialist. However, if you're building the infrastructure for an entire company, your architecture usually needs many types of infrastructure, and the general-purpose cloud providers will typically be a better fit, as they offer a one-stop-shop to meet all your needs.

For the examples in this book, the IaaS provider you'll be using is AWS, which offers a free tier and provides a huge range of reliable and scalable cloud services (servers, serverless, containers, databases, load balancers, etc.), so you can use it for most of the examples in this book. In addition, AWS is widely recognized as the dominant cloud provider—it has a 31% share of the market and has been the leader in the Gartner Magic Quadrant for the last 13 years—so it's something you're likely to use at work.

AN AWS ACCOUNT IS REQUIRED TO PROCEED!

This book includes examples that deploy into AWS. To run these examples, you need an AWS account you can authenticate to with administrator access. If you don't already have an AWS account, or you don't know how to authenticate to it, check out the "How to Authenticate to AWS with IAM Identity Center" tutorial on this book's website to learn how to create an AWS account (for free), create a user account with sufficient permissions, and authenticate to AWS on both the web and the command line.

To deploy the sample app in AWS, go through the following steps:

Step 1: Choose an AWS region

AWS has data centers all over the world, grouped into regions and availability zones. An *AWS region* is a separate geographic area, such as `us-east-2` (Ohio), `eu-west-1` (Ireland), and `ap-southeast-2` (Sydney). Within each region are multiple isolated data centers known as *availability zones* (AZs), such as `us-east-2a`, `us-east-2b`, and so on. Just about all the examples in this book will use the `us-east-2` (Ohio) region, so go into the AWS Console, and in the top right, pick `us-east-2` as the region to use, as shown in Figure 1-3. You may even want to configure `us-east-2` as your default region, so you don't accidentally end up in a different one. I've been using AWS for more than a decade, and I still end up in the wrong region by accident if I don't do this.

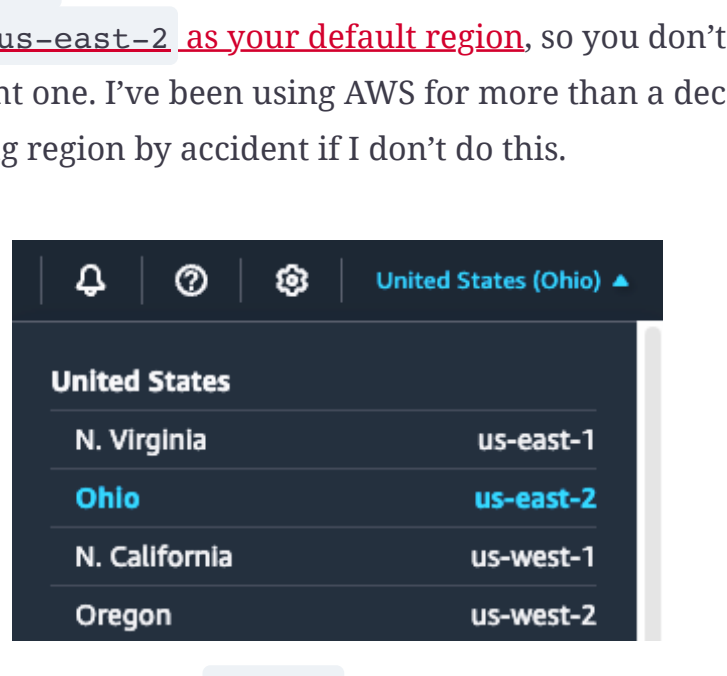
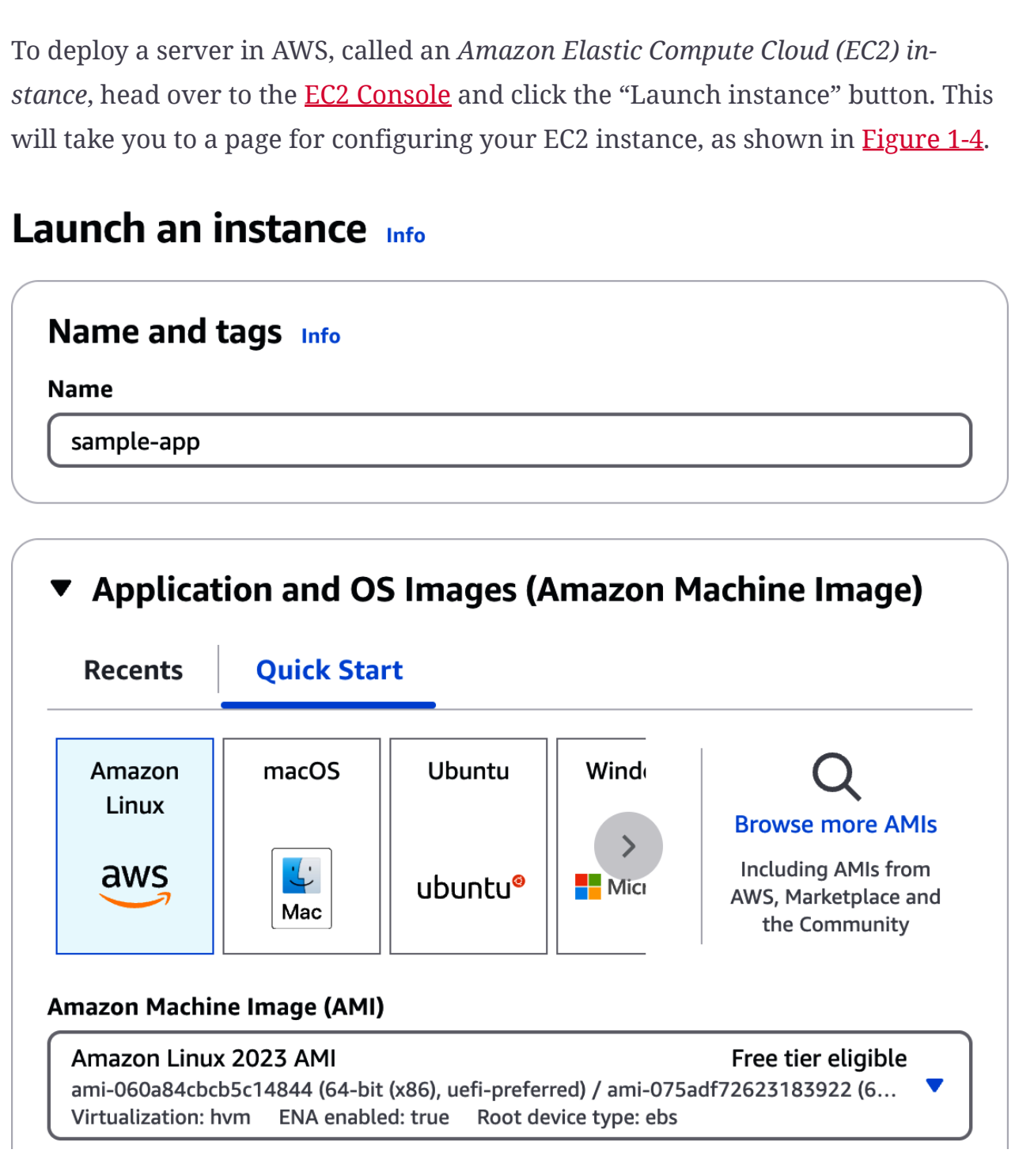


Figure 1-3. Pick `us-east-2` (Ohio) as your AWS region

Step 2: Deploy an EC2 instance

To deploy a server in AWS, called an *Amazon Elastic Compute Cloud (EC2) instance*, head over to the EC2 Console and click the "Launch instance" button. This will take you to a page for configuring your EC2 instance, as shown in Figure 1-4.



Launch an instance

Name and tags

Name: sample-app

Application and OS Images (Amazon Machine Image)

Amazon Machine Image (AMI)

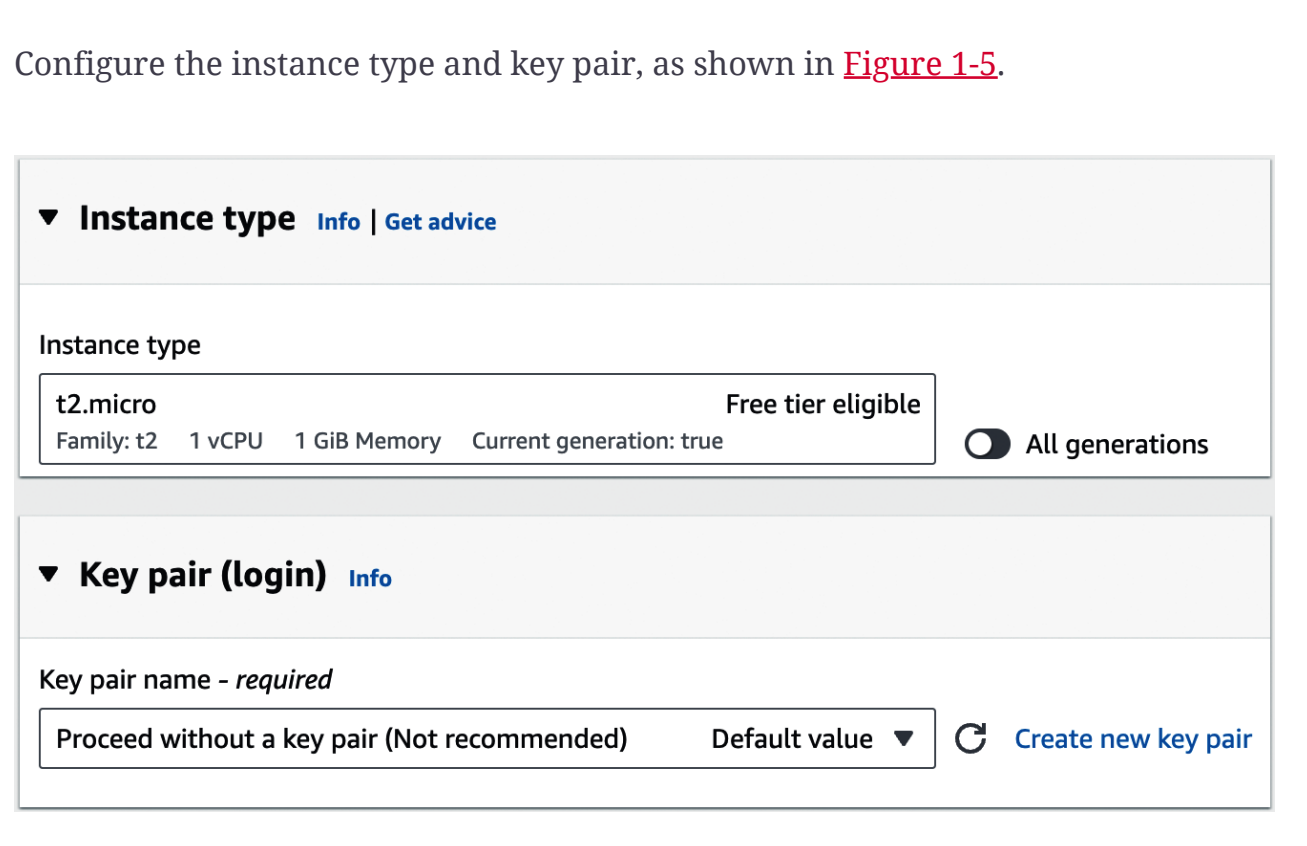
Amazon Linux 2023 AMI Free tier eligible
ami-060a84cbcb5c14844 (64-bit (x86), uefi-preferred) / ami-075adf72623183922 (6…
Virtualization: hvm ENA enabled: true Root device type: ebs

Figure 1-4. Configure the name and AMI to use for your EC2 instance

Fill in a name for the instance, such as "sample-app." Below that, you need to pick the *Amazon Machine Image* (AMI) to use, which specifies what OS and other software will be installed (you'll learn more about machine images in Chapter 2). For now, stick with the default, which should be Amazon Linux.

Step 3: Configure the EC2 instance

Configure the instance type and key pair, as shown in Figure 1-5.



Instance type

Instance type: t2.micro Free tier eligible
Family: t2 1 vCPU 1 GiB Memory Current generation: true

Key pair (login)

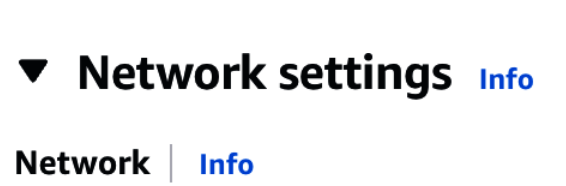
Key pair name - *required*: Proceed without a key pair (Not recommended) Default value

Figure 1-5. Configure the instance type and key pair to use for your EC2 instance

The *instance type* specifies the type of server to use: that is, what sort of CPU, memory, hard drive, etc. it'll have. For this quick test, you can use the default, which should be something like `t2.micro` or `t3.micro`, small instances (1 CPU, 1 GB of memory) that are part of the AWS free tier. The *key pair* can be used to connect to the EC2 instance via Secure Shell (SSH), a topic you'll learn more about in Chapter 7. You're not going to be using SSH for this example, so select "Proceed without a key pair."

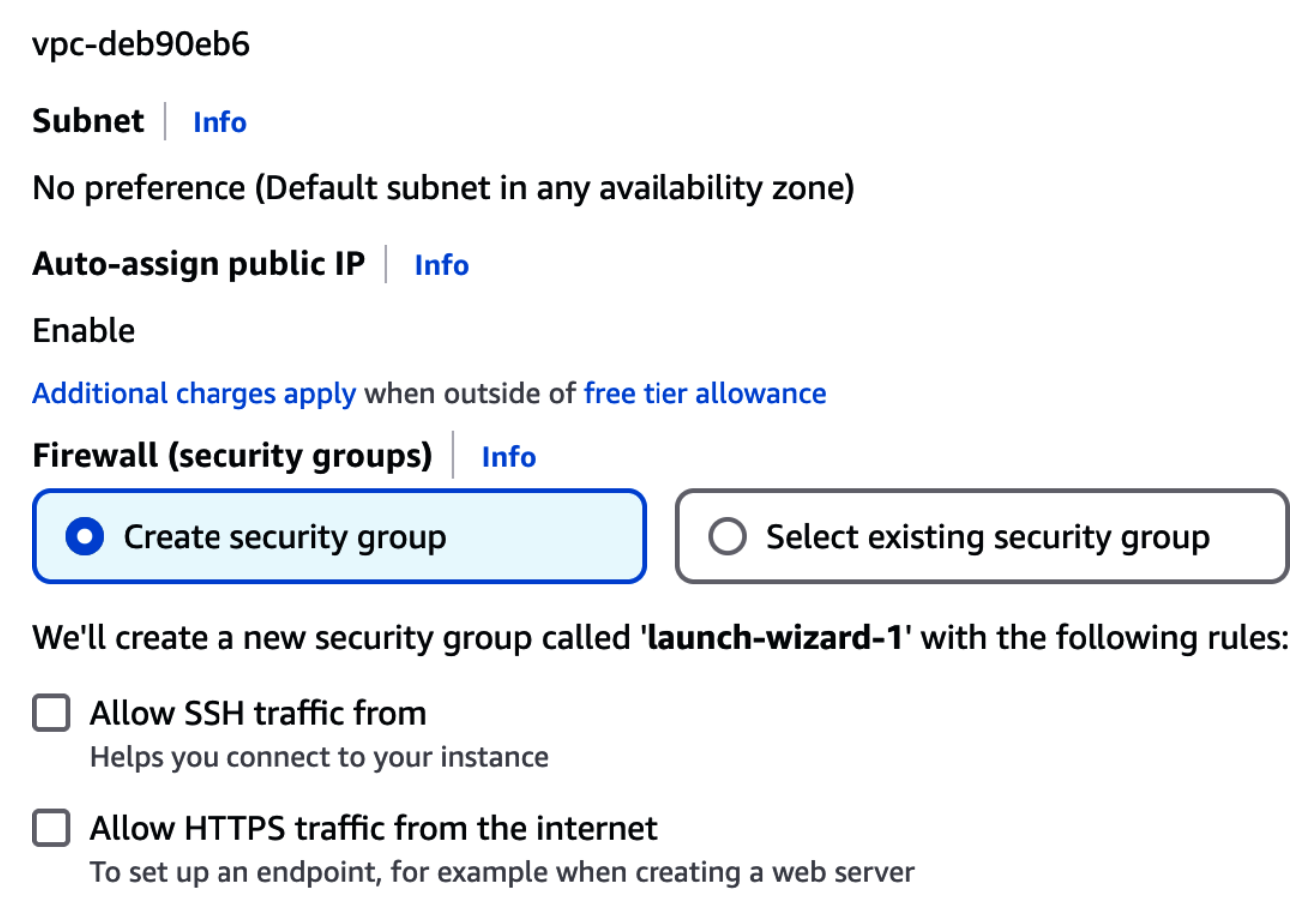
Step 4: Configure the network settings

Scroll down to the network settings, as shown in Figure 1-6. You'll learn about networking in Chapter 7. For now, you can leave most of these settings at their defaults: Network should be set to your default VPC (in Figure 1-6, my default VPC has the ID `vpc-deb90eb6`, but your ID will be different), Subnet should be set to "No preference," and "Auto-assign public IP" should be set to Enable. The only thing you should change is the "Firewall (security groups)" setting: select the "Create security group" radio button, disable the "Allow SSH traffic from" checkbox, and enable the "Allow HTTP traffic from the internet" checkbox, as shown in Figure 1-6. By default, EC2 instances have firewalls, called *security groups*, that don't allow any network traffic in or out. Allowing HTTP traffic tells the security group to allow inbound TCP traffic on port 80 so that the sample app can receive requests.



Network settings

Network


vpc-deb90eb6

Subnet | Info

No preference (Default subnet in any availability zone)

Auto-assign public IP | Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) | Info

● Create security group ○ Select existing security group

We'll create a new security group called 'launch-wizard-1' with the following rules:

☐ Allow SSH traffic from
Helps you connect to your instance

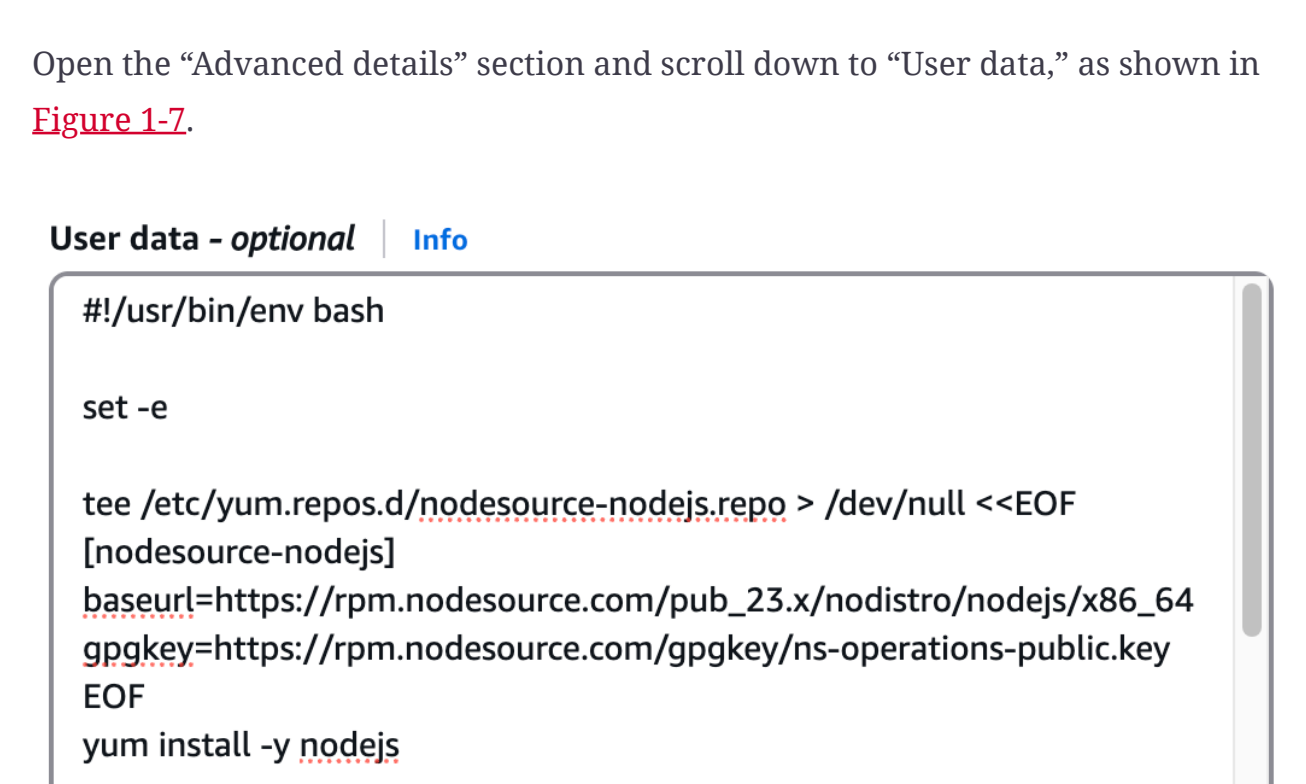
☐ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server

☑ Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server

Figure 1-6. Configure the network settings for your EC2 instance

Step 5: Configure advanced details

Open the "Advanced details" section and scroll down to "User data," as shown in Figure 1-7.


User data - *optional* | Info

```
#!/usr/bin/env bash

set -e

tee /etc/yum.repos.d/nodesource-nodejs.repo > /dev/null <<EOF
[nodesource-nodejs]
baseurl=https://rpm.nodesource.com/pub_23.x/nodistro/nodejs/x86_64
gpgkey=https://rpm.nodesource.com/gpgkey/ns-operations-public.key
EOF
yum install -y nodejs

tee app.js > /dev/null << "EOF"
const http = require('http');

const server = http.createServer((req, res) => {
```

Figure 1-7. Configure user data for your EC2 instance

User data is a script that the EC2 instance will execute the first time it boots up. Copy and paste the script shown in Example 1-2 into user data. You should also save a copy of this script in *ch1/ec2-user-data-script/user-data.sh* so you can reuse it later.

Example 1-2. User data script (*ch1/ec2-user-data-script/user-data.sh*)

```
#!/usr/bin/env bash

set -e

❶
tee /etc/yum.repos.d/nodesource-nodejs.repo > /dev/null <<EOF
[nodesource-nodejs]
baseurl=https://rpm.nodesource.com/pub_23.x/nodistro/nodejs/x86
gpgkey=https://rpm.nodesource.com/gpgkey/ns-operations-public.k
EOF
yum install -y nodejs

❷
tee app.js > /dev/null << "EOF"
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, World!\n');
});

❸
const port = process.env.PORT || 80;
server.listen(port,() => {
  console.log(`Listening on port ${port}`);
});
EOF

❹
nohup node app.js &
```

This user data script will do the following when the EC2 instance boots:

❶ Add the NodeSource repos to `yum` and use those repos to install Node.js.

❷ Write the sample app code to a file called *app.js*. This is the same Node.js code you saw earlier in the chapter, with one difference, as described in ❸.

❸ The only difference from the sample app code you saw earlier is that this code defaults to listening on port 80 instead of 8080, as that's the port you opened up in the security group.

❹ Run the app by using `node app.js`, just as you did on your own computer. Note the use of the ampersand (`&`), which runs the app in the background, so it doesn't block the user data script from exiting, and `nohup` ("no hangup"), which ensures the app keeps running even after the user data script exits and the terminal session ends.

WATCH OUT FOR SNAKES: THESE EXAMPLES HAVE SEVERAL PROBLEMS

The approach shown here with user data has several drawbacks, as explained in Table 1-2.

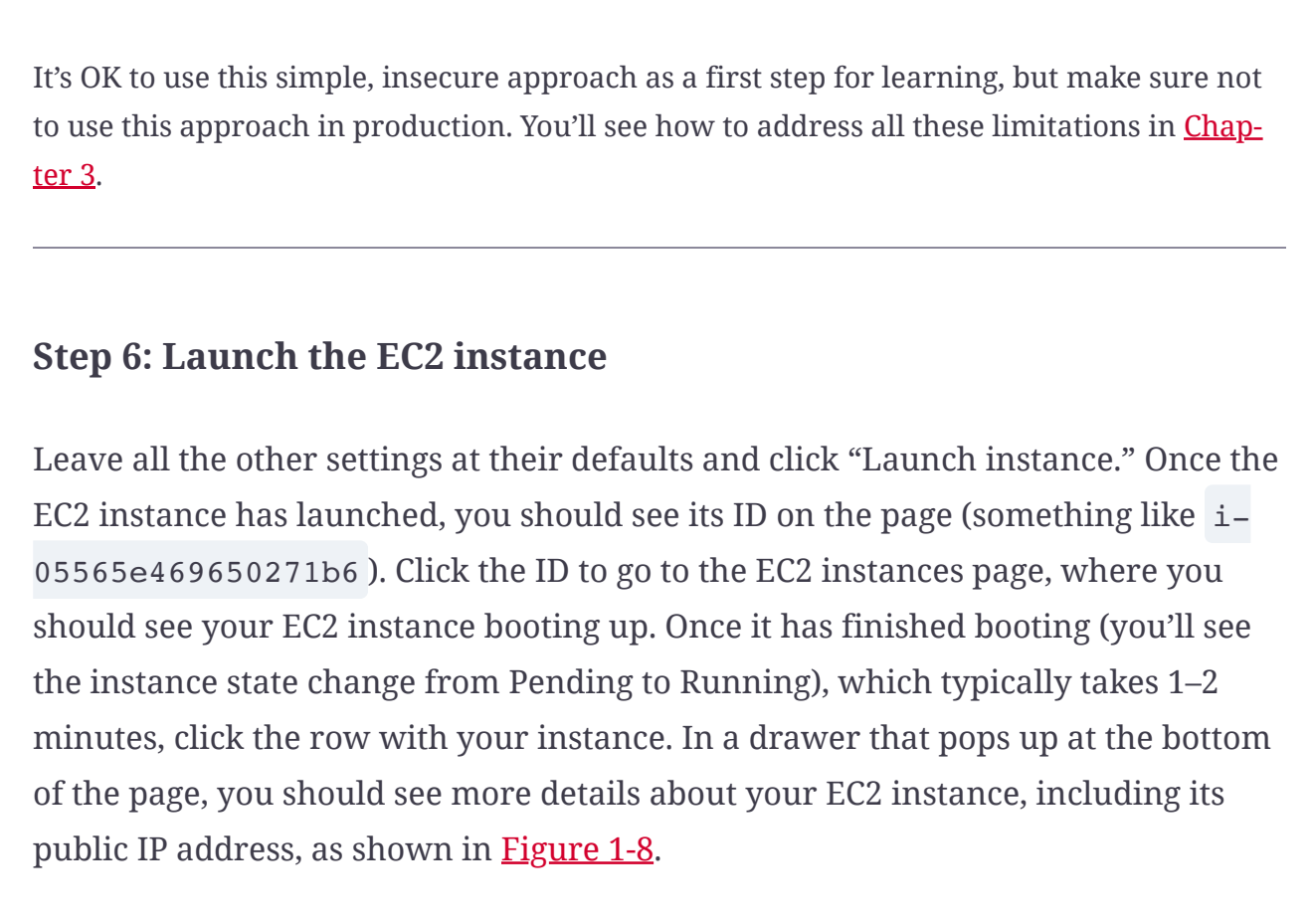
Table 1-2. Problems with the simplified example

Problem	What the example app does	What you should do instead
Root user	User data scripts run as the root user, so the Node.js app will end up running as the root user too.	Run apps using a separate OS user with limited permissions.
Port 80	The app listens on port 80, which requires root user permissions.	Have apps listen on a port greater than 1024.
User data limits	User data scripts are limited to 16 KB, so they aren't a great way to configure apps.	Use configuration management or server templating tools.
Process supervision	Nothing will restart the app if it crashes or the server reboots.	Use a process supervisor to monitor your app.
Node.js specifics	Runs one Node.js process (using one CPU core), in development mode.	Run one Node.js process per CPU core, in production mode.

It's OK to use this simple, insecure approach as a first step for learning, but make sure not to use this approach in production. You'll see how to address all these limitations in Chapter 3.

Step 6: Launch the EC2 instance

Leave all the other settings at their defaults and click "Launch instance." Once the EC2 instance has launched, you should see its ID on the page (something like `i-05565e469650271b6`). Click the ID to go to the EC2 instances page, where you should see your EC2 instance booting up. Once it has finished booting (you'll see the instance state change from Pending to Running), which typically takes 1–2 minutes, click the row with your instance. In a drawer that pops up at the bottom of the page, you should see more details about your EC2 instance, including its public IP address, as shown in Figure 1-8.



sample-app | i-05565e469650271b6 | Running

Instance: i-05565e469650271b6 (sample-app)

Instance ID: i-05565e469650271b6 (sample-app)

Public IPv4 address: 3.22.99.215 | open address

IPv6 address: –

Instance state: Running

Figure 1-8. Find the public IP address for your EC2 instance

Copy and paste that IP address, open *http://<IP>* in your browser (note: you have to actually type the *http://* portion in your browser, or the browser may try to use *https://* by default, which will *not* work), and you should see this:

```
Hello, World!
```

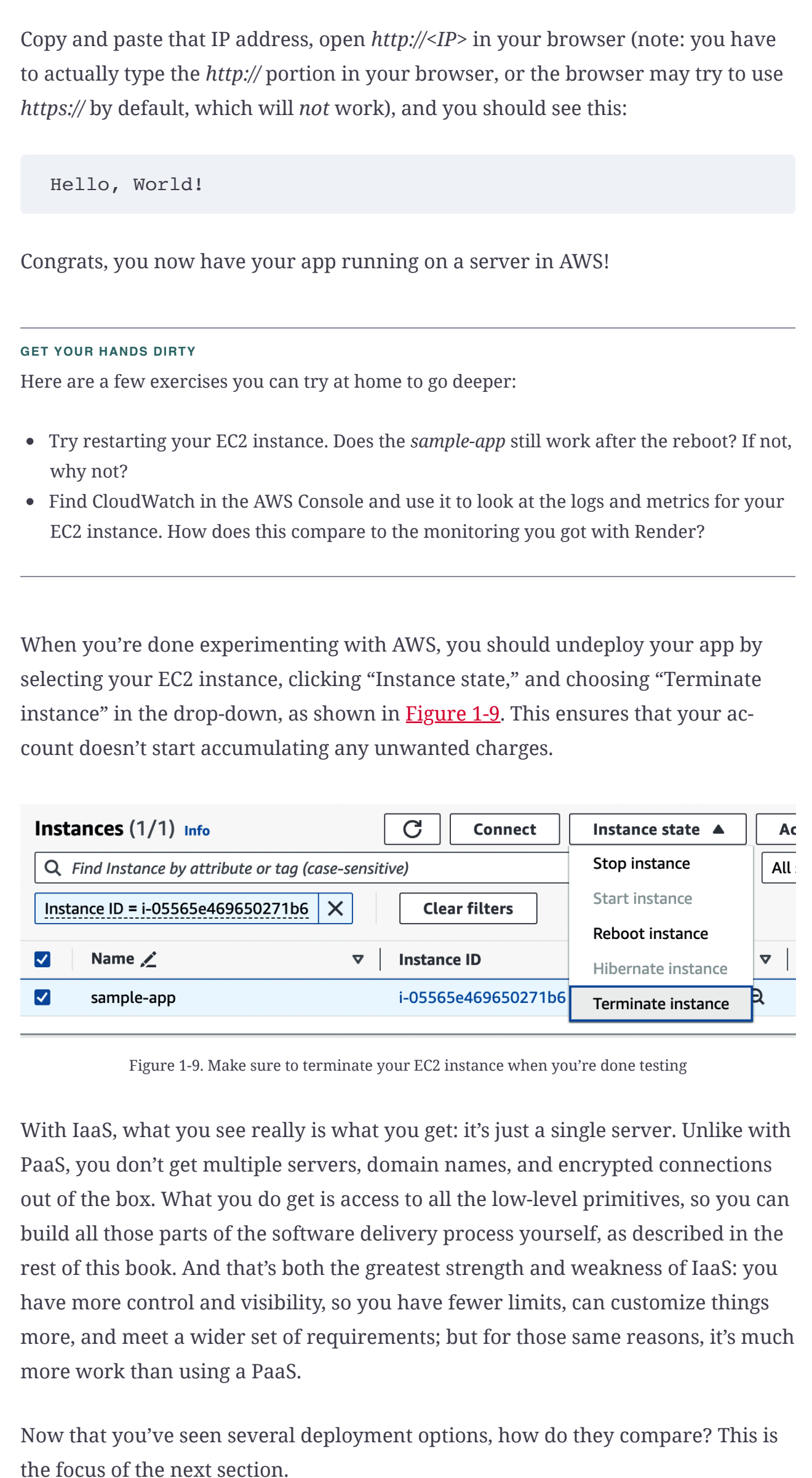
Congrats, you now have your app running on a server in AWS!

GET YOUR HANDS DIRTY

Here are a few exercises you can try at home to go deeper:

- Try restarting your EC2 instance. Does the *sample-app* still work after the reboot? If not, why not?
- Find CloudWatch in the AWS Console and use it to look at the logs and metrics for your EC2 instance. How does this compare to the monitoring you got with Render?

When you're done experimenting with AWS, you should undeploy your app by selecting your EC2 instance, clicking "Instance state," and choosing "Terminate instance" in the drop-down, as shown in Figure 1-9. This ensures that your account doesn't start accumulating any unwanted charges.



sample-app | i-05565e469650271b6

Figure 1-9. Make sure to terminate your EC2 instance when you're done testing

With IaaS, what you see really is what you get: it's just a single server. Unlike with PaaS, you don't get multiple servers, domain names, and encrypted connections out of the box. What you do get is access to all the low-level primitives, so you can build all those parts of the software delivery process yourself, as described in the rest of this book. And that's both the greatest strength and weakness of IaaS: you have more control and visibility, so you have fewer limits, can customize things more, and meet a wider set of requirements; but for those same reasons, it's much more work than using a PaaS.

Now that you've seen several deployment options, how do they compare? This is the focus of the next section.

Comparing Deployment Options

You've now seen several options for hosting your application; you could go with on prem or the cloud, and if you go with the cloud, you could go with IaaS or PaaS. This section compares these options, starting with on prem versus the cloud, followed by IaaS versus PaaS.

On Prem Versus the Cloud

When should you go with on prem, and when should you use the cloud? To start answering these questions, let's look at the key reasons to go with the cloud.

When to go with the cloud

If you're starting something new, in the majority of cases you should go with the cloud. Here are just a few of the advantages:

Elasticity and pay as you go

> On prem, you pay up-front for capacity that may go unused; for example, if you need 10 servers most of the time but anticipate traffic spikes, you may have to buy 50 servers. The cloud offers *pay-as-you-go* pricing, which starts out cheap or free, increases only with usage, and allows you to scale *elastically*; for example, you pay for 10 servers most of the time, and pay for 50 servers only while there's a traffic spike.

Speed

> Getting new hardware takes weeks on prem but just minutes in the cloud.

Maintenance and expertise

> Data centers require a lot of expertise (in hardware, cooling, and power) and maintenance (replacing broken or obsolete equipment), all of which the cloud handles for you.

Managed services

> With the cloud, you get not only servers but also services such as managed databases, load balancers, file stores, networking, analytics, and machine learning.

Security

> Despite the myth that on prem is more secure, the world's most secure data centers belong to the cloud providers; for example, AWS complies with 143 security standards (e.g., PCI DSS, HIPAA, and NIST 800-171) and has dozens of third-party audits and attestations (e.g., SOC, ISO, and FedRAMP).

Global reach

> The cloud gives you instant access to dozens of data centers around the world.

Scale

> Major cloud providers can invest more than almost anyone else in data centers. For example, AWS made $107 billion in 2024—and it's still growing.

For all these reasons, the cloud is the de facto option for most new startups, as well as new projects in many established companies.

KEY TAKEAWAY 2

Using the cloud should be your *default* choice for most new deployments these days.

With all these advantages of using the cloud, does it ever make sense to use on-prem?

When to go with on prem

Running servers yourself is the better option in the following cases:

You already have an on-prem presence

> If your company already has its own data centers and they are working well for you, stick with them! If it ain't broke, don't fix it.

You have usage patterns that are a better fit for on prem

> Certain usage patterns may be a better fit for on prem—for instance, steady, predictable usage that doesn't benefit from elasticity (see Basecamp for an example) or usage that requires lots of bandwidth (bandwidth can be expensive in the cloud).

You have compliance requirements that are a better fit for on prem

> You may find that some compliance standards, regulations, laws, auditors, and customers have not yet adapted to the cloud, so depending on your industry and product, you may find that on prem is a better fit.

You need more control over pricing

> Although competition between the major cloud vendors has historically driven prices down, every now and then a vendor will abruptly raise prices.[1] One way to minimize this risk is to pick a cloud vendor with a good track record related to pricing and to negotiate a long-term contract. Another option is to avoid the cloud entirely and go on prem. The only option I wouldn't recommend is using the cloud but trying to avoid "lock-in," as described in the following "The Cloud Vendor Lock-in Myth" sidebar.

THE CLOUD VENDOR LOCK-IN MYTH

Many companies worry about *vendor lock-in*, where a cloud vendor suddenly increases pricing, but you can't do anything about it, because it's too expensive to migrate all your infrastructure. A persistent myth says the way to prevent this sort of lock-in is to use only *generic services* (e.g., virtual servers) and to avoid any of the cloud vendor's *proprietary services* (e.g., managed databases, file stores, machine learning tools), which will make it "easy" to migrate to another vendor's generic services. In most cases, this is a waste of time.

If you use only generic services, you'll end up creating your own bespoke versions of the proprietary services, so your initial deployment will take *x* months longer. If you have to migrate later, in theory, you won't have to rewrite your code to work with the proprietary services of the new vendor, which might save you *y* months, and in the best-case scenario, *y* > *x*. However, in practice, using proprietary services lets you launch considerably faster (*x* is large), and you typically have to make major changes to your bespoke tooling to get it to work in another deployment environment (*y* is small), so you'd be lucky to break even (*y* = *x*).

That means you're paying a guaranteed higher cost *x* up front to *maybe* save *y* time later, which is usually a bad trade-off (a lower *expected value*, in probability terms). Unless you end up migrating many times, you're typically better off picking the best tooling available (e.g., if you're deploying to the cloud, using the cloud vendor's proprietary services) and getting live as quickly as possible.

It's worth mentioning that it doesn't have to be cloud versus on prem; it can also be cloud *and* on prem, as discussed next.

When to go with hybrid

In a *hybrid* deployment, you use a mixture of cloud and on-prem. The most common use cases for this are the following:

Partial cloud migration

> Some companies migrate a subset of their apps (including all new apps) to the cloud, but they keep some apps on prem, either temporarily (as a full migration can take years) or permanently (some apps are not worth the cost to migrate).

Right tool for the job

> You may choose to use the cloud when it's a good fit (e.g., an app that needs elasticity) and to use on prem when it's a good fit (e.g., an app with steady traffic patterns). Use the right tool for the job!

My goal is to allow you to try out the examples in this book as quickly and cheaply as possible, so the cloud is the right tool for this job. Therefore, while the underlying concepts will apply to both the cloud and on prem, most of the examples in this book use the cloud. And if you're going to use the cloud, you need to know when to use IaaS versus PaaS.

IaaS Versus PaaS

If you're using the cloud, when should you go with IaaS, and when should you go with PaaS? To start answering this question, let's look at the key reasons to go with PaaS.

When to go with PaaS

This may seem like a strange thing to say in a book about DevOps and software delivery, but if you can create a great product without having to invest much in DevOps and software delivery, *that's a good thing*. Your customers don't care what kind of deployment pipeline you have, or whether you are running a fancy Kubernetes cluster or the newest type of database. All that matters is that you can create a product that meets your customers' needs.

KEY TAKEAWAY 3

You should spend as little time on software delivery as you possibly can while still meeting your company's requirements.

If you can find someone else who can take care of software delivery for you and still meet your requirements, you should take advantage of that as much as possible. And that's precisely what a good PaaS offers: out-of-the-box software delivery. If you can find a PaaS that meets your requirements, you should use it, stick with it for as long as you can, and avoid having to re-create all those software delivery pieces until you absolutely have to. PaaS is a good choice in these cases:

Side projects

> If you're working on a side project, the last thing you want to do is kill your passion for that project by spending all your time fighting with builds or pipelines or networking. Instead, let a PaaS do the heavy lifting.

Startups and small companies

> If you're building a new company, you should almost always start with a PaaS. Startups are a race against time: can you build something the market wants before you run out of money? As you saw earlier in this chapter, you can get live on a PaaS in *minutes*, and for most startups, the scalability, availability, security, and compliance needs are minimal, so you can keep running on a PaaS for years before you run into the limitations. It's only when you find product/market fit and start hitting the problem of having to scale your company—which is a good problem to have—that you may need to move off PaaS.

New and experimental projects

> If you're at an established company that has a slow software delivery process, using a PaaS can be a great way to quickly try out new and experimental projects, especially if those projects don't have the same scalability, availability, security, and compliance needs as your company's more mature products.

As a general rule, you want to use a PaaS whenever you can, and move on to IaaS only when a PaaS can no longer meet your requirements, as per the next section.

When to go with IaaS

In the following cases, an IaaS is usually a better fit:

Load

> If you're dealing with a lot of traffic, PaaS pricing may become prohibitively expensive. Moreover, PaaS usually limits the types of apps and architectures you can use, so you may have to migrate to IaaS to scale your systems.

Company size

> As you shift from a handful of developers to dozens of teams with hundreds of developers, not only can PaaS pricing become untenable, but you may also hit limits with governance and access controls (e.g., allowing some teams to make some types of changes but not others).

Availability

> Your business may need to provide uptime guarantees that are higher than what your PaaS can provide. Moreover, when your app has an outage or a bug, PaaS offerings are often limited in the type of visibility and connectivity options they provide (e.g., Heroku doesn't let you connect to servers over SSH), so you may have to migrate to IaaS to improve your ability to debug and introspect your systems.

Security and compliance

> One of the most common reasons to move off PaaS is that most of them (with the notable exception of Aptible) do not provide sufficient visibility, access, or control to meet security and compliance requirements (e.g., SOC 2, ISO 27001, PCI DSS).

You go with IaaS whenever you need more control, more performance, and/or more security. If your company gets big enough, one or more of these needs will likely push you from PaaS to IaaS; that's just the price of success.

KEY TAKEAWAY 4

Go with PaaS whenever you can; go with IaaS when you have to.

Now that you've had a chance to do a basic app deployment and compare hosting options, the question is, what happens next? How will your architecture and processes change as your company grows? This is the topic of the next section.

The Evolution of DevOps

While writing my first book, *Hello, Startup* (O'Reilly), I interviewed early employees from some of the most successful companies of the last 20 years, including Google, Facebook, LinkedIn, Twitter, GitHub, Stripe, Instagram, Pinterest, and many others. One thing that struck me is that the architecture and software delivery processes at just about every one of these software companies evolved along similar lines. They had individual differences here and there, but far more similarities than differences, and the broad shape of the evolution repeated again and again. In this section, I share this evolutionary process, broken into nine high-level steps.

If you're new to DevOps and software delivery, you may be unfamiliar with some of the terms used here. Don't panic. The idea is to start with a top-down overview—a bit like a high-level map—to help you understand the various ingredients and how they fit together. You can think of this content as a high-level preview of the topics you'll cover in the following chapters. As you go through each chapter, you'll zoom in on each of these topics, study each one in detail, and try most of them out with real examples. You can then zoom back out and revisit this high-level map at any time to see the big picture and get your bearings again.

Let's begin with step 1, as shown in Figure 1-10, which is where most projects start, including new startups, new initiatives at established companies, and side projects.

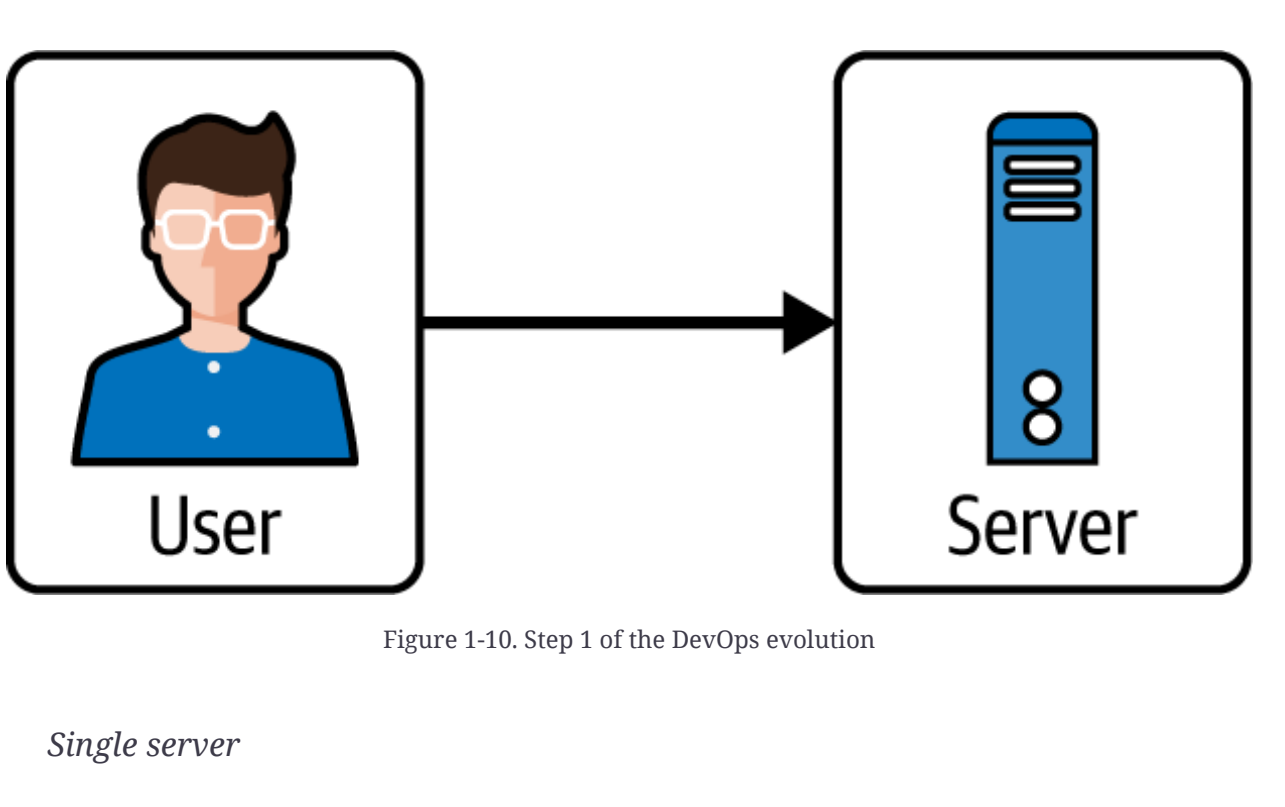

User
Server

Figure 1-10. Step 1 of the DevOps evolution

Single server

All your application code runs on a single server.

ClickOps

You manage all your infrastructure and deployments manually.

Does this sound familiar? It's what you just did earlier in this chapter, using Render and AWS. So congrats, you've completed step 1! But this is only the beginning. As traffic and team size grow, you move on to step 2, shown in Figure 1-11.

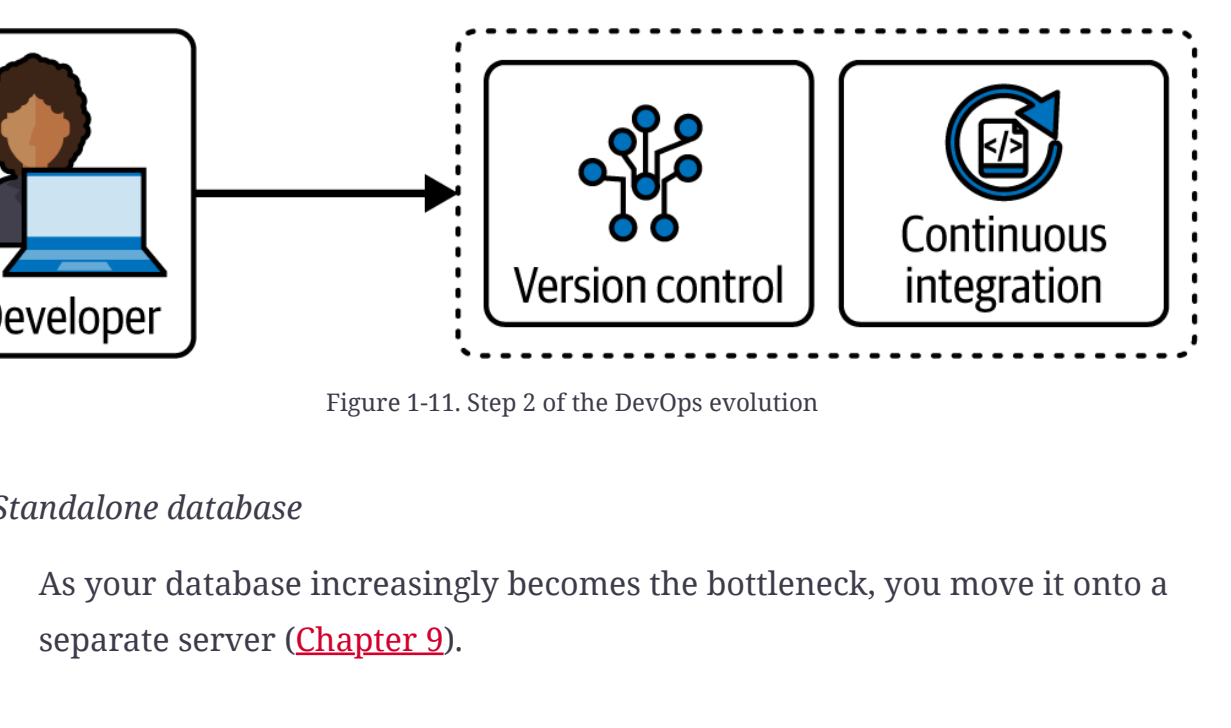

User
Server
Database
Developer
Version control
Continuous integration

Figure 1-11. Step 2 of the DevOps evolution

Standalone database

As your database increasingly becomes the bottleneck, you move it onto a separate server (Chapter 9).

Version control

As your team grows, you use a version-control system to collaborate on your code and track all changes (Chapter 4).

Continuous integration

To reduce bugs and outages, you set up automated tests (Chapter 4) and continuous integration (Chapter 5).

As traffic continues to grow, you move on to step 3, shown in Figure 1-12.

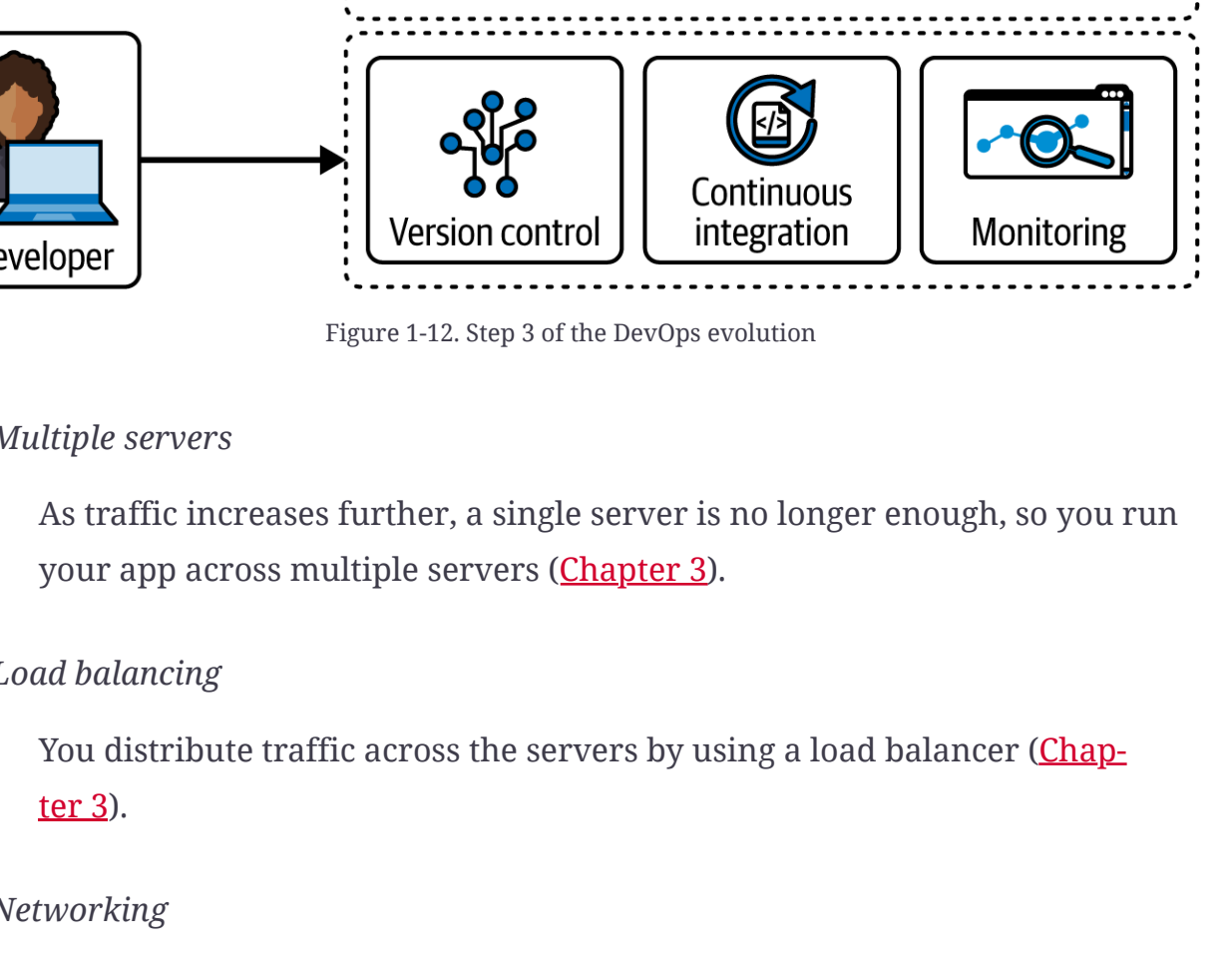

VPC
Backups
User
Load balancer
Servers
Database
Developer
Version control
Continuous integration
Monitoring

Figure 1-12. Step 3 of the DevOps evolution

Multiple servers

As traffic increases further, a single server is no longer enough, so you run your app across multiple servers (Chapter 3).

Load balancing

You distribute traffic across the servers by using a load balancer (Chapter 3).

Networking

To protect your servers, you put them into a private network (Chapter 7).

Data management

You set up schema migrations and backups for your data stores (Chapter 9).

Monitoring

To get better visibility into your systems, you set up monitoring (Chapter 10).

Most software projects never need to make it past these first three steps. If you're one of them, don't fret: this is a good thing. The first three steps are relatively simple. The technologies involved are fast to learn, easy to set up, and fun to work with. If you're forced into the subsequent steps, it's because you're facing new problems that require more-complex architectures and processes to solve, and this additional complexity has a considerable cost. If you aren't facing those problems, you can, and should, avoid that cost.

That said, larger, more established companies, with more users, may have to move on to step 4, shown in Figure 1-13.

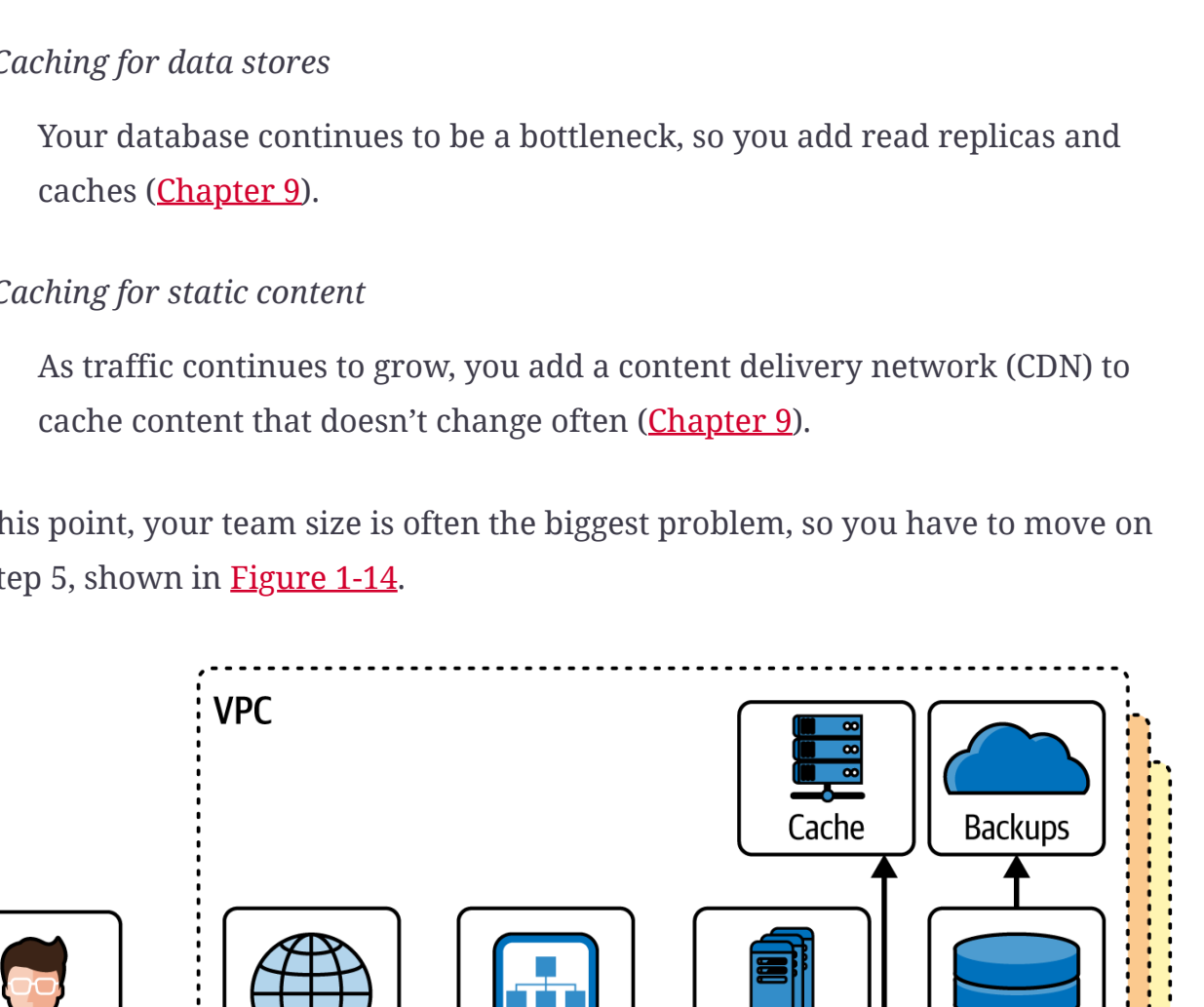

VPC
Cache
Backups
User
CDN
Load balancer
Servers
Database
Developer
Version control
Continuous integration
Monitoring

Figure 1-13. Step 4 of the DevOps evolution

Caching for data stores

Your database continues to be a bottleneck, so you add read replicas and caches (Chapter 9).

Caching for static content

As traffic continues to grow, you add a content delivery network (CDN) to cache content that doesn't change often (Chapter 9).

At this point, your team size is often the biggest problem, so you have to move on to step 5, shown in Figure 1-14.

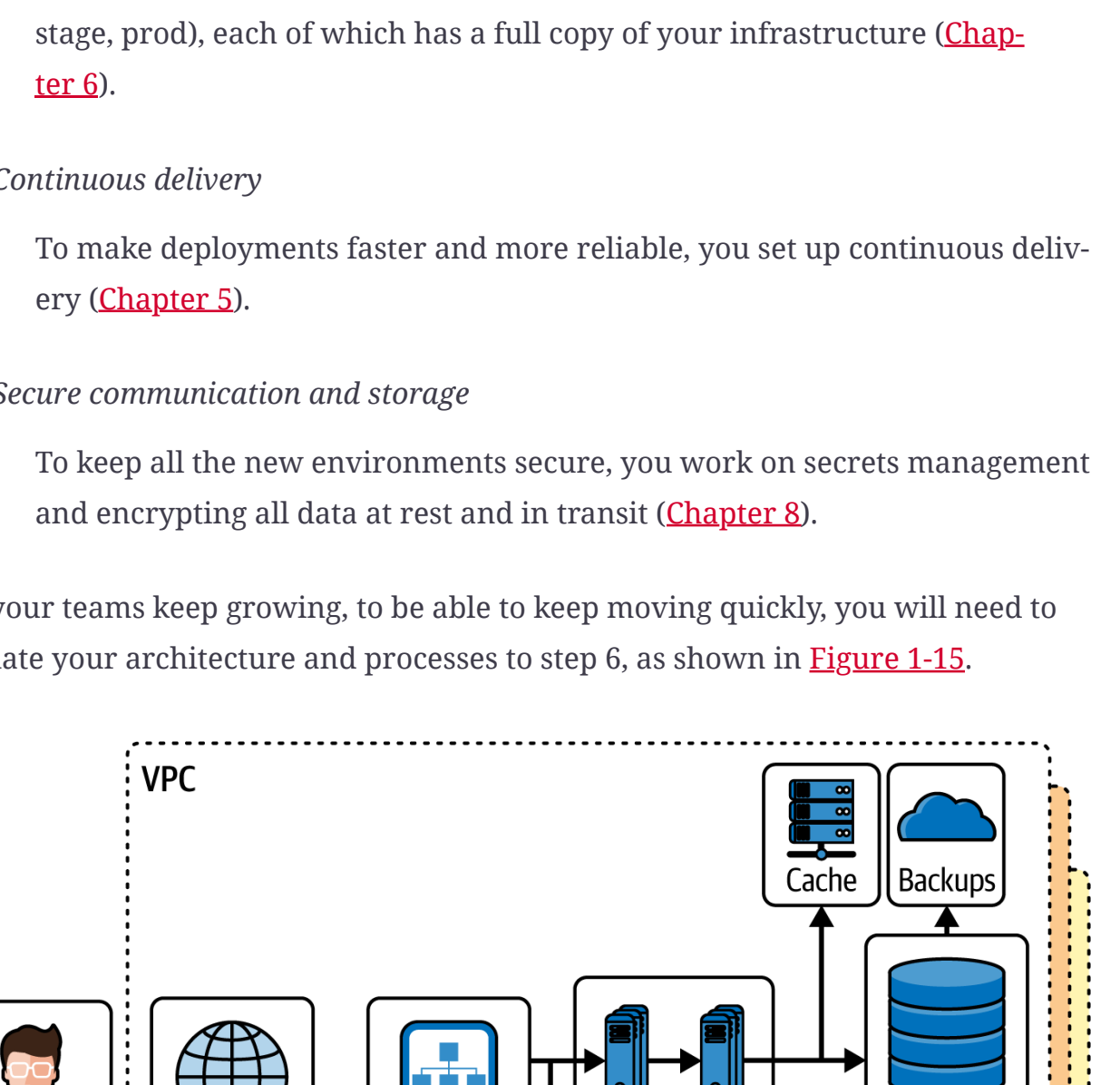

VPC
Cache
Backups
User
CDN
Load balancer
Servers
Database
Prod
Stage
Dev
Developer
Version control
Continuous integration
Continuous delivery
Monitoring
Encryption, secrets

Figure 1-14. Step 5 of the DevOps evolution

Multiple environments

To help teams do better testing, you set up multiple environments (e.g., dev, stage, prod), each of which has a full copy of your infrastructure (Chapter 6).

Continuous delivery

To make deployments faster and more reliable, you set up continuous delivery (Chapter 5).

Secure communication and storage

To keep all the new environments secure, you work on secrets management and encrypting all data at rest and in transit (Chapter 8).

As your teams keep growing, to be able to keep moving quickly, you will need to update your architecture and processes to step 6, as shown in Figure 1-15.

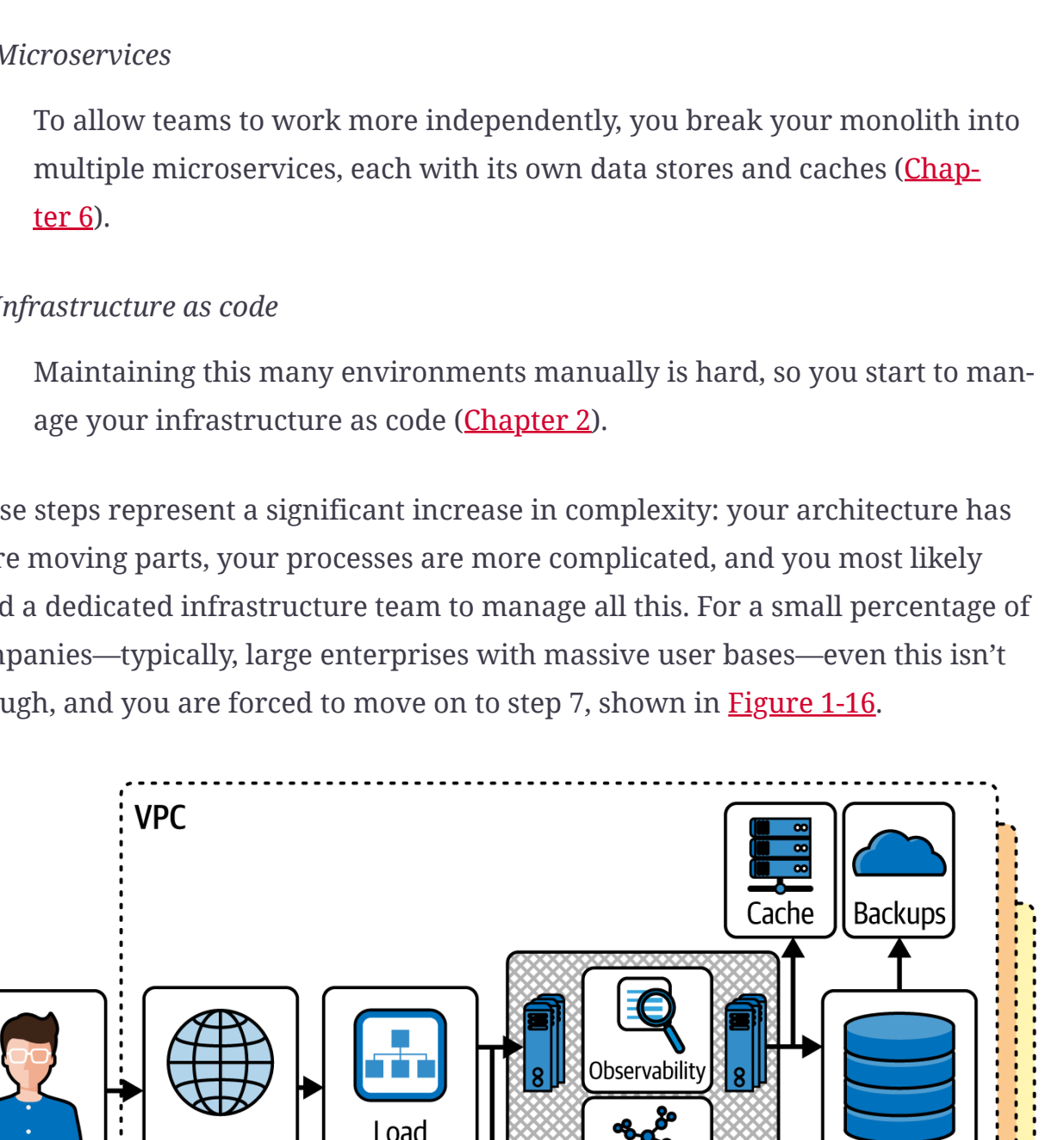

VPC
Cache
Backups
User
CDN
Load balancer
Database
Microservices
Database
Cache
Prod
Stage
Dev
Developer
Version control
Continuous integration
Continuous delivery
Monitoring
Encryption, secrets
Infrastructure as code

Figure 1-15. Step 6 of the DevOps evolution

Microservices

To allow teams to work more independently, you break your monolith into multiple microservices, each with its own data stores and caches (Chapter 6).

Infrastructure as code

Maintaining this many environments manually is hard, so you start to manage your infrastructure as code (Chapter 2).

These steps represent a significant increase in complexity: your architecture has more moving parts, your processes are more complicated, and you most likely need a dedicated infrastructure team to manage all this. For a small percentage of companies—typically, large enterprises with massive user bases—even this isn't enough, and you are forced to move on to step 7, shown in Figure 1-16.

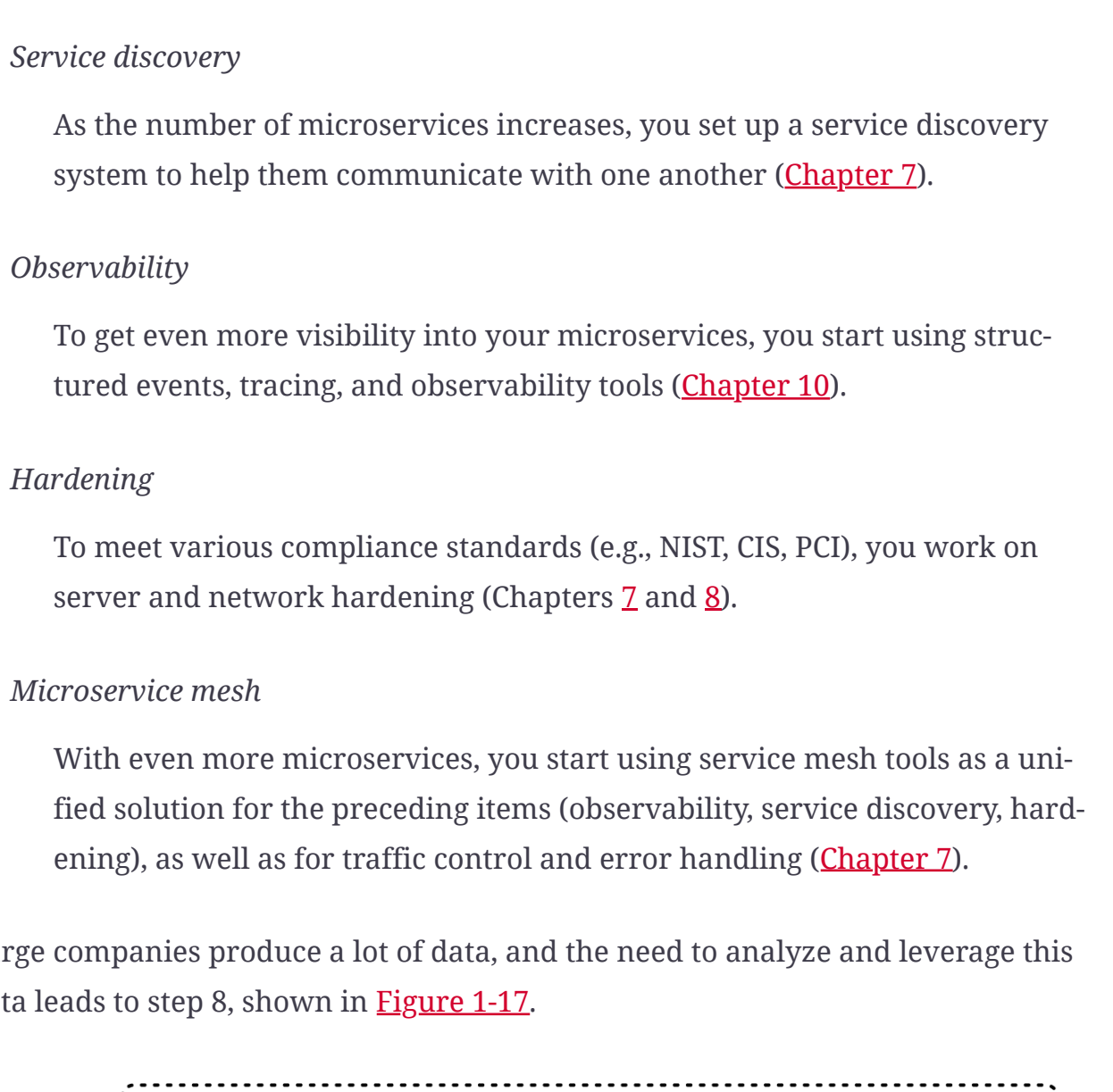

VPC
Cache
Backups
User
CDN
Load balancer
Observability
Service discovery
Hardening
Microservices mesh
Database
Database
Cache
Prod
Stage
Dev
Developer
Version control
Continuous integration
Continuous delivery
Monitoring
Encryption, secrets
Infrastructure as code

Figure 1-16. Step 7 of the DevOps evolution

Service discovery

As the number of microservices increases, you set up a service discovery system to help them communicate with one another (Chapter 7).

Observability

To get even more visibility into your microservices, you start using structured events, tracing, and observability tools (Chapter 10).

Hardening

To meet various compliance standards (e.g., NIST, CIS, PCI), you work on server and network hardening (Chapters 7 and 8).

Microservice mesh

With even more microservices, you start using service mesh tools as a unified solution for the preceding items (observability, service discovery, hardening), as well as for traffic control and error handling (Chapter 7).

Large companies produce a lot of data, and the need to analyze and leverage this data leads to step 8, shown in Figure 1-17.

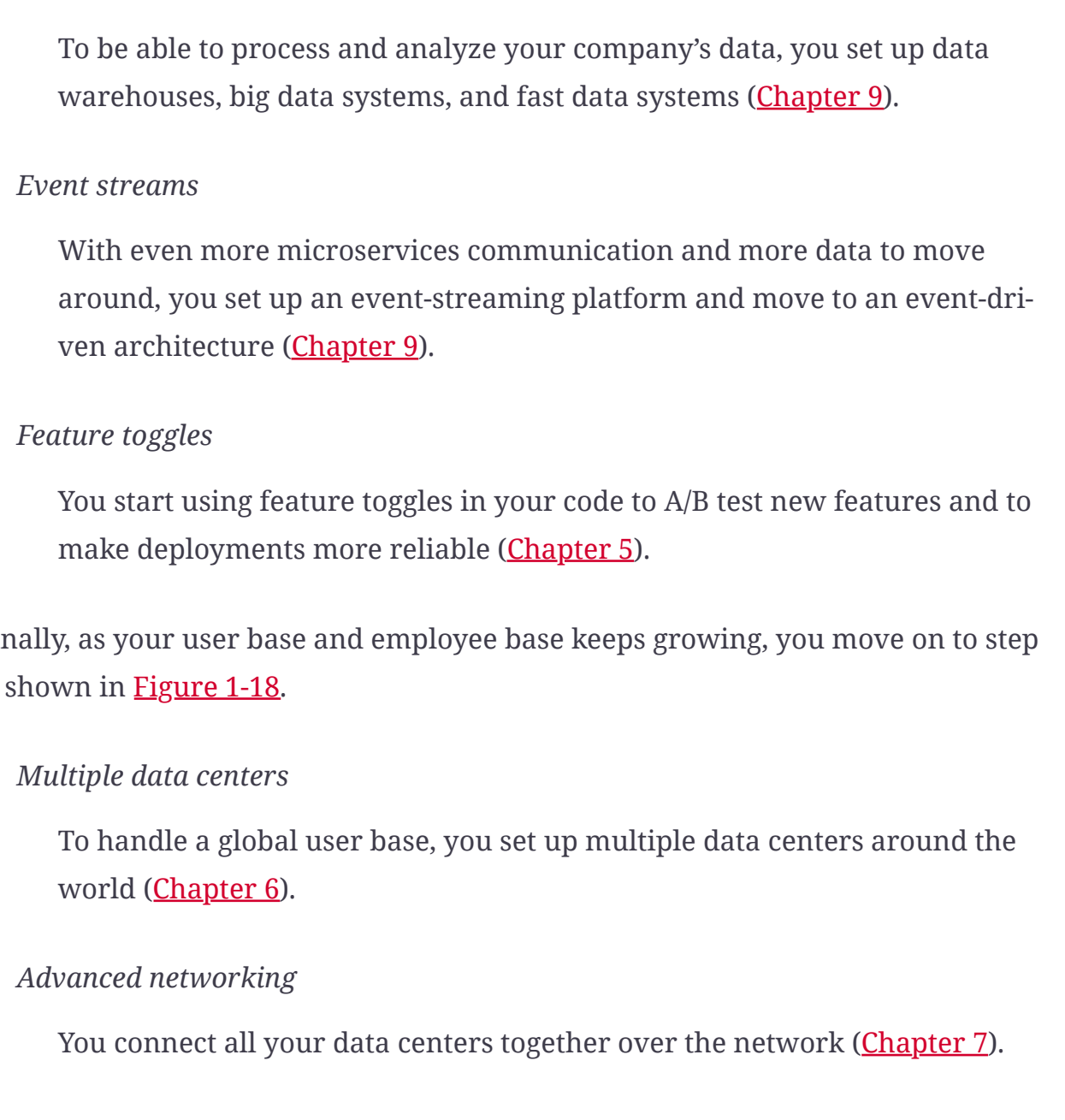

VPC
Cache
Backups
User
CDN
Load balancer
Observability
Service discovery
Hardening
Microservices mesh
Database
Database
Data warehouse
Cache
Event-streaming platform
Prod
Stage
Dev
Developer
Version control
Continuous integration
Continuous delivery
Monitoring
Encryption, secrets
Infrastructure as code
Feature toggles

Figure 1-17. Step 8 of the DevOps evolution

Analytics tools

To be able to process and analyze your company's data, you set up data warehouses, big data systems, and fast data systems (Chapter 9).

Event streams

With even more microservices communication and more data to move around, you set up an event-streaming platform and move to an event-driven architecture (Chapter 9).

Feature toggles

You start using feature toggles in your code to A/B test new features and to make deployments more reliable (Chapter 5).

Finally, as your user base and employee base keeps growing, you move on to step 9, shown in Figure 1-18.

Multiple data centers

To handle a global user base, you set up multiple data centers around the world (Chapter 6).

Advanced networking

You connect all your data centers together over the network (Chapter 7).

Internal developer platform

To help boost developer productivity and to standardize coding practices, you set up an internal developer platform (Chapter 11).

These last three steps are for companies that face the toughest problems and have to deal with the most complexity: global deployments, thousands of developers, millions of customers. Even the architecture you see in step 9 is still a simplification compared to what the top 0.1% of companies face, but if that's where you're at, you'll need more than this introductory book!

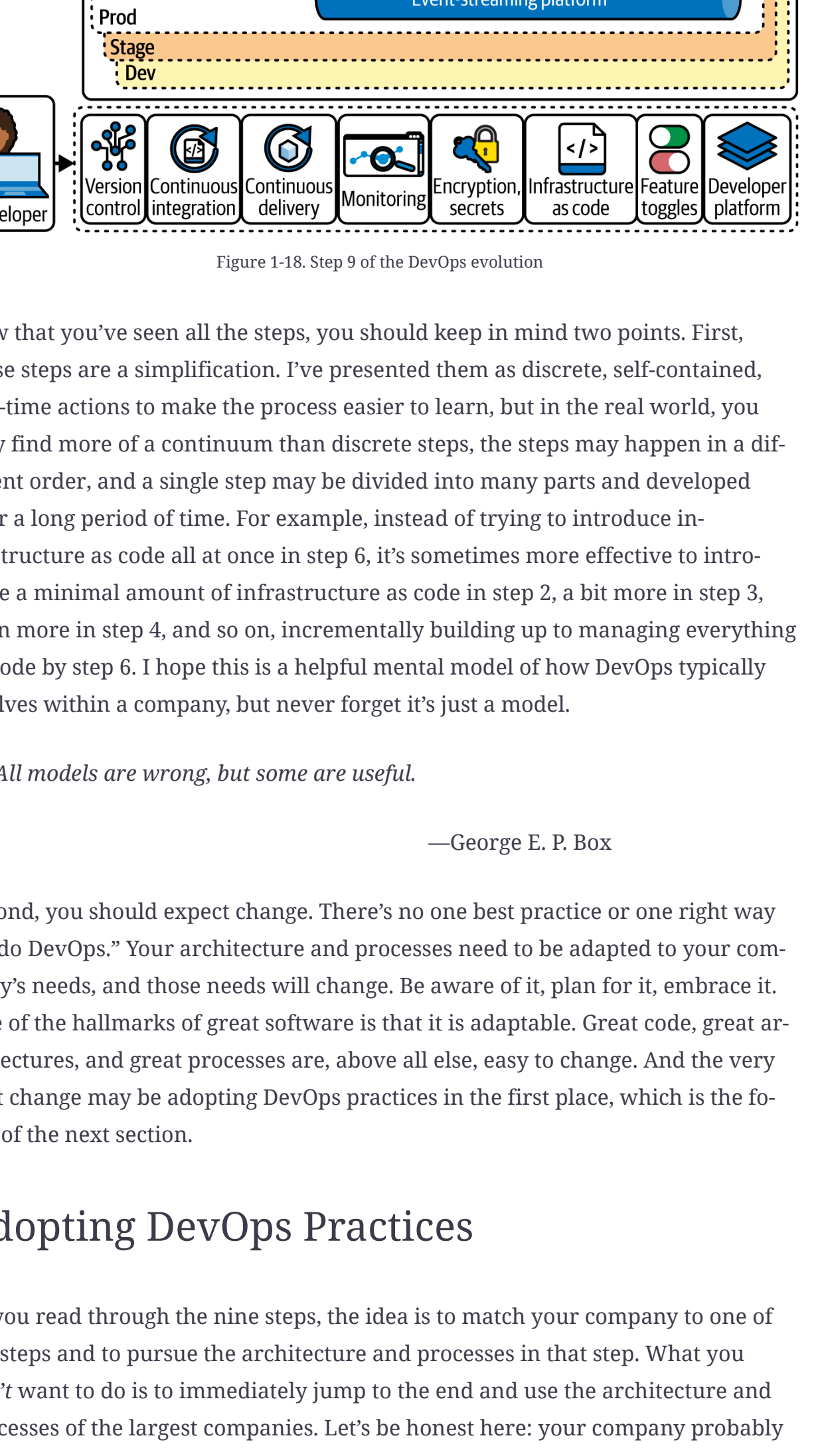

Data center #1
VPC
Microservices mesh
User
CDN
Load balancer
Observability
Service discovery
Hardening
Cache
Backups
Database
Data warehouse
Cache
Database
Event-streaming platform
Prod
Stage
Dev
VPN
Data center #2
VPC
Microservices mesh
User
CDN
Load balancer
Observability
Service discovery
Hardening
Cache
Backups
Database
Data warehouse
Cache
Database
Event-streaming platform
Prod
Stage
Dev
Developer
Version control
Continuous integration
Continuous delivery
Monitoring
Encryption, secrets
Infrastructure as code
Feature toggles
Developer platform

Figure 1-18. Step 9 of the DevOps evolution

Now that you've seen all the steps, you should keep in mind two points. First, these steps are a simplification. I've presented them as discrete, self-contained, one-time actions to make the process easier to learn, but in the real world, you may find more of a continuum than discrete steps, the steps may happen in a different order, and a single step may be divided into many parts and developed over a long period of time. For example, instead of trying to introduce infrastructure as code all at once in step 6, it's sometimes more effective to introduce a minimal amount of infrastructure as code in step 2, a bit more in step 3, even more in step 4, and so on, incrementally building up to managing everything as code by step 6. I hope this is a helpful mental model of how DevOps typically evolves within a company, but never forget it's just a model.

> *All models are wrong, but some are useful.*
>
> —George E. P. Box

Second, you should expect change. There's no one best practice or one right way to "do DevOps." Your architecture and processes need to be adapted to your company's needs, and those needs will change. Be aware of it, plan for it, embrace it. One of the hallmarks of great software is that it is adaptable. Great code, great architectures, and great processes are, above all else, easy to change. And the very first change may be adopting DevOps practices in the first place, which is the focus of the next section.

Adopting DevOps Practices

As you read through the nine steps, the idea is to match your company to one of the steps and to pursue the architecture and processes in that step. What you *don't* want to do is to immediately jump to the end and use the architecture and processes of the largest companies. Let's be honest here: your company probably isn't Google or Netflix; you don't have the same scale, you don't have the same problems to solve, and therefore, the same solutions won't be a good fit. In fact, adopting the solutions for a different stage of company may be actively harmful. Every time I see a three-person startup running an architecture with 12 microservices, Kubernetes, a service mesh, and an event-streaming platform, I just shake my head; they're paying a massive cost to solve problems they don't have.

KEY TAKEAWAY 5

Adopt the architecture and software delivery processes that are appropriate for your stage of company.

Even if you are a massive company, you still shouldn't try to adopt every DevOps practice all at once. One of the most important lessons I've learned in my career is that most large software projects fail. Whereas roughly 3 out of 4 small IT projects (less than $1 million) are completed successfully, only 1 out of 10 large projects (greater than $10 million) are completed on time and on budget, and more than one-third of large projects are never completed at all.[2]

This is why I get worried when I see the CEO or CTO of a large company give marching orders that everything must be migrated to the cloud, the old data centers must be shut down, and that everyone will "do DevOps" (whatever that means), all within six months. I'm not exaggerating when I say that I've seen this pattern several dozen times, and without exception, *every single one of these initiatives has failed*. Inevitably, two to three years later, every one of these companies is still working on the migration, the old data center is still running, and no one can tell whether they are really "doing DevOps" or not.

A more effective way to accomplish any large migration project is to do it incrementally. The key to *incrementalism* is not just splitting the work into a series of small steps, but splitting it in such a way that every step brings its own value—even if the later steps never happen. To understand why this is so important, consider the opposite—suppose that you have a huge migration project, broken into the following steps:

1. Redesign the UI.
2. Rewrite the backend.
3. Migrate the data.

You complete the first step, but you can't launch the UI because it relies on the new backend in the second step. So, next, you rewrite the backend, but you can't launch that either, until you migrate the data in the third step. Only when you complete all three steps can you get any value from this work. This is *false incrementalism*: you've split the work into smaller steps, but you get value only when all the steps are completed.[3] This is a huge risk, as conditions change all the time, and the project may be paused or cancelled or modified partway through, before you've completed all the steps. If that happens, you get the worst possible outcome: you've invested a bunch of time and money but got nothing in return.

Instead, you want each part of the project to deliver value so that even if the entire project doesn't finish, no matter what step you completed, it was still worth doing. You can accomplish this by focusing on solving one, small, concrete problem at a time. For example, instead of trying to do a "big-bang" migration to the cloud, try to identify one team that is struggling and then work to migrate just that team. If you can get a quick win by fixing one real, concrete problem right away and making one team successful, you'll begin to build momentum. This will allow you to go for another quick win, and another one after that, incrementally working through all the parts of the larger migration. But even if the larger migration is canceled, at least one team is more successful now, so it was still worth the investment.

KEY TAKEAWAY 6

Adopt DevOps incrementally, as a series of small steps, where each step is valuable by itself.

Conclusion

You now know the basics of deploying apps. Here are the six key takeaways from this chapter:

You should never expose apps running on a personal computer to the outside world

Instead, deploy those apps on a server.

Using the cloud should be your default choice for most new deployments these days

Run your server(s) in the cloud whenever you can. Use on-prem only if you already have an on-prem presence or you have load patterns or compliance requirements that work best on-prem.

You should spend as little time on software delivery as you possibly can while still meeting your company's requirements

If you can offload your software delivery to someone else while still responsibly meeting your company's requirements, you should.

Go with PaaS whenever you can; go with IaaS when you have to

A PaaS lets you offload most of your software delivery needs, so if you can find a PaaS that meets your company's requirements, you should use it. Go with an IaaS only if your company's requirements exceed what a PaaS can offer.

Adopt the architecture and software delivery processes that are appropriate for your stage of company

Always pick the right tools for the job.

Adopt DevOps incrementally, as a series of small steps, where each step is valuable by itself

Avoid big-bang migrations and false incrementalism, which provide value only after all steps have been completed.

One way to summarize these ideas is the concept of *minimum effective dose*. This is a term from pharmacology, where you use the smallest dose of medicine that will give you the biological response you're looking for. That's because just about every drug, supplement, and intervention becomes toxic at a high enough dose, so you want to use just enough to get the benefits, and no more. The same is true with DevOps: every architecture, process, and tool has a cost, so you want to use the simplest and most minimal solution that gives you the benefits you're looking for, and no more. Don't use a fancy architecture or software delivery process if a simpler one will do; instead, always aim for the minimum effective dose of DevOps.

Knowing how to deploy your apps is an important step, but it's just the first step in understanding DevOps and software delivery. Quite a few steps remain. One problem you may have noticed, for example, is that you had to deploy everything in this chapter by manually clicking around a web UI. Doing things manually is tedious, slow, and error prone. Imagine if instead of one app, you had to deploy ten, and you had to do it many times per day. Not fun.

The solution is automation, and managing your infrastructure as code, which is the topic of Chapter 2.

---

[1] For example, Google Cloud's 2022 price changes increased storage pricing by as much as 50%, and Vercel's 2024 price changes increased pricing for some users by as much as 10 times.

[2] The Standish Group, "CHAOS Manifesto 2013: Think Big, Act Small," 2013, *https://oreil.ly/XGCzJ*.

[3] Dan Milstein, "How to Survive a Ground-Up Rewrite Without Losing Your Sanity," OnStartups, April 8, 2013, *https://oreil.ly/lAeBz*.